Scenario 01:

Scenario Name: ***Search Functionality***
Weak Prompt: *Test the search feature.*

What I Feel is Missing

Application type is not specified: It does not clarify whether the search feature belongs to an e-commerce website, hospital application, or another type of system.

Filter and sorting requirements are missing: There is no information about available filters, sorting options, or their expected behaviour.

Empty and invalid input scenarios are not mentioned: The prompt does not specify how to handle empty searches, invalid characters, special characters, or unexpected input.

Performance testing requirements are missing: There is no mention of response-time expectations, concurrent users, load conditions, or search performance benchmarks.

✅ **Rewritten Prompt**

Persona:
Act as a Senior QA Engineer with expertise in functional testing.

Context:
The application has a search feature that allows users to search for products by name. Search results are displayed in a list.

Instruction:
Test the search feature by identifying positive, negative, boundary, and edge-case scenarios. Verify search accuracy, partial matches, case sensitivity, empty input, invalid input, and handling of no results.

Example:
If the user searches for "iPhone", the application should display matching products containing "iPhone" in their names.

Output Format:
Provide the test cases in a table with the following columns: Test Case ID, Scenario, Test Data, Steps, Expected Result, and Test Type.

Tone:
Use a professional, clear, and technically precise tone.

Observations:

The detailed prompt improved the output by making it more structured, specific, and comprehensive. Instead of providing only general observations about search results, it

generated 47 test cases covering positive, negative, boundary, and edge-case scenarios, with test data, execution steps, and expected results. It also included security, performance, and error-handling scenarios, making the output more actionable for test execution and automation.

Scenario 02:

Scenario Name: ***File Upload***
Weak Prompt: Generate test cases for file upload.

What I Feel is Missing

The prompt does not identify the supported file formats, such as PDF, JPG, PNG, or DOCX, or the file types that must be rejected.

There is no information about the maximum permitted file size or how the application should handle files that exceed the limit.

The prompt does not explain what should happen when a valid file is uploaded, an invalid file is selected, or an upload fails.

There is no instruction to test malicious files, disguised file extensions, or other file-upload security risks.

✅ **Rewritten Prompt**

Persona

Act as a Lead QA Engineer with expertise in functional testing.

Context

The application provides a file-upload feature that allows users to upload documents and images.

- *Supported file types: PDF, JPG, and PNG.*
- *Maximum file size: 10 MB per file.*
- *Valid files should be uploaded successfully, and a confirmation message should be displayed.*

- *Unsupported file types and files exceeding the size limit should be rejected with appropriate error messages.*
- *The application should validate uploaded files securely and prevent malicious files from being accepted or executed.*

Instruction

Generate comprehensive test cases for the file-upload feature, covering:

1. *Positive scenarios for valid file types and valid file sizes.*
2. *Negative scenarios for unsupported file types, empty files, corrupted files, and files exceeding the size limit.*
3. *Boundary-value scenarios for files just below, exactly at, and just above the maximum size.*
4. *Error handling for interrupted uploads, upload failures, and repeated upload attempts.*
5. *Security scenarios involving malicious files, misleading file extensions, and file-type validation.*
6. *Verification of success messages, error messages, uploaded file details, and the final upload status.*

Do not assume security protections or expected behavior beyond the stated requirements. Identify any additional requirements that need clarification.

Example

- *Input: A valid PDF file of 5 MB.*
  Expected result: The file is uploaded successfully, and a confirmation message is displayed.
- *Input: A 12 MB PDF file.*
  Expected result: The upload is rejected, and an appropriate size-limit error message is displayed.
- *Input: An executable file renamed with a .pdf extension.*
  Expected result: The file is rejected if its actual content does not meet the application's file-validation and security requirements.

Output Format

Present the test cases in a table with the following columns:

Test Case ID | Test Scenario | Test Data | Preconditions | Test Steps | Expected Result | Test Type (Positive/Negative/Boundary/Security)

Organize the test cases by functional area and avoid duplicate scenarios.

Tone

Use a professional, clear, concise, and technically precise tone.

Observations

The detailed prompt improved the output by making it more comprehensive, structured, and actionable. Compared to the generic one-line prompt, it generated detailed test cases covering positive and negative scenarios, file type and size validation, boundary conditions, security, error handling, UI/UX, accessibility, performance, and browser compatibility. It also included test data, preconditions, execution steps, expected results, and clarification questions for ambiguous requirements. Overall, the detailed prompt produced better test coverage and a more practical starting point for test execution and automation.

Scenario 03:

Scenario Name: ***Shopping Cart***
Weak Prompt: Test the cart functionality.

Weak Prompt

Test the cart functionality.

What I Feel is Missing

Cart operations are not specified: The prompt does not mention adding items to the cart, removing items, or updating product quantities.

Price calculation validation is missing: There is no information about validating subtotal, quantity-based totals, taxes, shipping charges, or the final cart amount.

Coupon and discount scenarios are not covered: The prompt does not specify how to test valid and invalid coupons, discount application, expiry, or discount removal.

Out-of-stock handling is not defined: There is no guidance on how the cart should behave when a product becomes unavailable or its stock is insufficient.

✅ **Rewritten Prompt**

Persona:

Act as a Senior QA Engineer with expertise in e-commerce functional testing, integration testing, and end-to-end testing.

Context:

The application is an e-commerce website where users can add products to their shopping cart, update quantities, remove items, and apply coupons or discounts. The cart displays product details, quantities, prices, discounts, and the total payable amount. Product availability and stock quantities may change.

Instruction:

Generate comprehensive BDD test scenarios for the shopping cart functionality, covering the following areas:

1. *Add, Remove, and Update Products: Verify adding products to the cart, removing items, updating quantities, and handling duplicate products.*
2. *Price Calculation Validation: Verify product prices, subtotals, quantity-based totals, discounts, taxes, shipping charges, and the final payable amount, wherever applicable.*
3. *Coupon and Discount Handling: Test valid, invalid, expired, and ineligible coupons, as well as applying, removing, and recalculating discounts.*
4. *Out-of-Stock Handling: Verify how the cart behaves when a product is out of stock, inventory is insufficient, or a product becomes unavailable after being added to the cart.*
5. *Empty Cart and Boundary Scenarios: Test empty-cart behavior, minimum and maximum quantities, quantity limits, and invalid quantity values.*
6. *Cart Persistence and Consistency: Verify that cart contents and calculated totals remain consistent after page refreshes and navigation between pages.*
7. *Error Handling: Verify appropriate messages and recovery behavior when cart operations fail.*

For each scenario:

- *Write the test case in BDD format using Given, When, and Then.*
- *Use And where additional preconditions, actions, or expected outcomes are required.*
- *Ensure that each scenario has clear preconditions, actions, and verifiable expected results.*

- *Include positive, negative, and boundary scenarios where applicable.*
- *Avoid duplicate scenarios and keep each scenario focused on one behavior.*
- *Do not invent business rules that are not specified. Explicitly identify requirements that need clarification.*

Example:

Scenario: Verify that a user can add a product to the cart and see the correct subtotal

Given the user is on the product details page
And the product is available in stock at a price of ₹500
When the user adds the product to the cart with a quantity of 2
Then the product should appear in the cart with a quantity of 2
And the subtotal should be ₹1,000 before applicable taxes, shipping charges, or discounts
And the cart should display the correct product details and price

Output Format:

Present all test cases in BDD format using the following structure:

- *Feature: Shopping Cart Functionality*
- *Scenario: A clear description of the behaviour being tested*
- *Given: The initial context and preconditions*
- *And: Additional preconditions, if required*
- *When: The user action or event*
- *And: Additional actions, if required*
- *Then: The expected outcome*
- *And: Additional expected outcomes, if required*

Organize the scenarios under relevant functional areas, such as cart operations, price calculations, coupons, inventory, and persistence.

Use a separate BDD scenario for each distinct test condition.

[Do not] present the test cases in a table.

Tone:

Use a professional, clear, concise, and technically precise tone. Write scenarios that are easy for QA engineers, developers, and business stakeholders to understand and execute.

Observations

The detailed prompt significantly improved the output by transforming a general request for cart testing into a comprehensive, structured set of 72 BDD test scenarios. It covers positive, negative, integration, boundary, persistence, and error-handling cases, with clear Given–When–Then steps, expected results, sample prices, and business-rule clarifications. Unlike the generic response, which only listed possible testing areas and asked for a website URL or codebase, the detailed output provides an actionable test plan that can guide test design and execution.